

XRPL Validator — Daily Engineering Report • 2026-09-07

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **36 commits** (xrpl-validator 36)
- **28 engine findings fixed** (F198–F226)
- 3 test / vector commits
- Amendment **fixCleanup3_3_0** held majority (since 2026-08-28 11:15 UTC); activation due ~2026-09-11 11:15 UTC

Engine findings fixed

- **fix(xrpl-ledger): F198 — the direct walk's tail pool turn iterates like rippled's flow()** (c1b1e51 , xrpl-validator) rippled's flow() keeps iterating a single-path pool: each pass is one AMMLiquidity::getOffer — maxOffer, 99% of the pool's out at the CURRENT balances — and the next pass starts from the moved pool, until the taker's in is spent, the want is met, or a pass misses limitQuality (limitOut sizes it to the limit, the realised quality is judged, "Path rejected by limitQuality" → "All strands dry"). Our tail turn ran ...
- **fix(xrpl-ledger): F199 — TrustSet refuses tfSetNoRipple on a negative balance** (79395eb , xrpl-validator) TrustSet::doApply: "Cannot set noRipple on a negative balance" — if (bSetNoRipple && !bClearNoRipple) { if ((bHigh ? saHighBalance : saLowBalance) >= beast::zero) uFlagsOut |= (bHigh ? lsfHighNoRipple : lsfLowNoRipple); else return tecNO_PERMISSION; } the first refusal on an existing line, ahead of the reserve check. The sender's balance is the stored (low-perspective) one negated for the high side. do_apply set the ...
- **fix(xrpl-ledger): F200 — an in-limited pool slice delivers no more than the remaining want** (a8439a9 , xrpl-validator) consume's plain in-limit (take_in > rem_gets → re-swap the whole remaining in through swapAssetIn) can land the out ABOVE the remaining want. rippled's forward pass never delivers that surplus: the strand's last DirectStep keeps its REVERSE cache whenever the forward input exceeds it (setCacheLimiting , finding 170), so the beneficiary is credited the reverse out — the want — while the pool parts with the whole ...
- **fix(xrpl-ledger): F201 — consecutive self-offers are swept in the pass that is still walking** (7c49aa8 , xrpl-validator) BookStep::forEachOffer runs tryAMM ONCE per pass, at the FIRST tip's quality — if (tryAMM(offers.tip().quality())) do { execOffer(tip) } while (offers.step()) — and execOffer removes the taker's own offer before it looks at any amount (limitSelfCrossQuality). So once a pass has removed the self-offer at one level, it steps straight onto the next level's tip: a self-offer there is removed too, and no pool offer ...
- **fix(xrpl-ledger): F202 — the bridged walk's remaining want is the entry want less the ascending fold of its saved outs** (5182de9 , xrpl-validator) rippled's StrandFlow keeps the per-iteration outs in a flat_multiset and re-derives remainingOut = outReq - sum(savedOuts) after every iteration (StrandFlow.h:743) — the saved outs

folded ASCENDING, each += an STAmount add canonicalised to sixteen digits. The direct walk has carried that since finding 173; the bridged walk still subtracted each fill's out from the running remainder.

- **fix(xrpl-ledger): F206 — a pool slice's quality is getRate-rounded, as rippled's Quality{amounts} (7aaf827 , xrpl-validator)** An AMM fib offer's quality is rippled's `Quality{amounts} = getRate(out, in) = divide(in, out) : floor(in·1017/out) + 5`, then canonicalised to sixteen digits under Number's default ToNearest. `slice_rate` divided to nearest instead — one ulp apart on the rate's last digit — and a leg fill limited by the other leg (`ceilInStrict`: the input divided by that rate, floored) inherits the ulp.
- **fix(xrpl-ledger): F205 — a bridge admitted by its multipath bound runs only if its single-path bound clears the limit too (2b27ffd , xrpl-validator)** `flow()`'s `activateNext` prices a strand's `qualityUpperBound` under the CURRENT `multiPath` flag. A round that admits the bridge as one of two strands prices it under `multiPath` — leg A by the pool's fib slice at its spot, leg B by its tip — but once that verdict leaves the bridge alone, the next `activateNext` re-prices it under `multiPath=false`: leg A by the pool offer ANCHORED on the USD/XRP tip, its inside transfer rate ...
- **fix(xrpl-ledger): F204 — an input-limited payment strand drives its middle hops by the carry (888ecec , xrpl-validator)** Once the reverse pass asks more input than the strand has, rippled's forward pass is driven by the INPUT at every book step — `BookStep::fwdImp` hands `limitStepIn` the whole carry, and the reverse amounts are reinstated only on an exact cache hit — and only the closing `DirectStep` clamps the delivery to the reverse want (finding 170). Each hop here bought exactly the reverse want and left the carry's rounding ...
- **fix(xrpl-ledger): F203 — a payment strand's quality function takes a pool only where its fee-inclusive spot beats the tip (955de48 , xrpl-validator)** rippled's `limitOut` (`StrandFlow.h`) composes each step's quality function, and a `BookStep`'s is the pool's (linear) only when `tipOfferQualityF` gets an AMM offer that beats the tip: `getAMMOffer(view, qualityThreshold(lob))`, where a PAYMENT step's `qualityThreshold` is the tip itself (`BookPaymentStep::qualityThreshold { return lobQuality; }`, `BookStep.cpp:320`). The `qualityThreshold_ > lobQuality → nullopt ...`
- **fix(xrpl-ledger): F208 — a crossing's pool turn is bounded by the taker's live line (bdc467e , xrpl-validator)** rippled's `DirectStepI` reverse pass bounds the taker's in by `PaymentSandbox::balanceHookIOU = min(live line, original - Σdebits)`. The direct walk's `rem_gets` is the second figure — the fold of the spends — and the line, adjusted sequentially at sixteen digits, can sit an ulp or two under it. The CLOB fill already takes the line (finding 174); the pool turn sized its slice by `rem_gets` alone.
- **fix(xrpl-ledger): F209 — an IOU difference at Number's cusp rounds the coarse mantissa (f0642ec , xrpl-validator)** rippled 3.3.0 subtracts IOUs through `Number` (`STAmount.cpp operator+ → IOUAmount::operator+= → Number::operator+=`) at the sixteen-digit "Small" mantissa scale — `Rules.cpp setCurrentTransactionRules` selects the nineteen-digit scales only under `SingleAssetVault` or `LendingProtocol`, neither live on mainnet — where the cusp-rounding fix is Disabled. The operand with the smaller exponent is truncated to the ...
- **fix(xrpl-ledger): F210 — a forward pool hop re-anchors to the reverse pass's cached out (a2b29db , xrpl-validator)** rippled's `BookStep::fwdImp`, when "the step produced more output in the forward pass than the reverse pass while consuming the same input (or less)", re-prices the offer for the cached out — `limitStepOut(cache_→ out)`, for a single-path pool `AMMOffer::limitOut = swapAssetOut` — and when the input that requires equals the input provided it consumes that input and produces exactly the output the reverse pass ...
- **fix(xrpl-ledger): F211 — a dry rival strand hands the round to the direct pool (b5d35c7 , xrpl-validator)** An AMM offer is gated by ITS OWN book's CLOB tip alone: `AMMLiquidity::getOffer(sb, clobQuality)`

returns `nullopt` when `Quality{amounts} < clobQuality`, and `clobQuality` is the tip of the book the step walks (`tryAMM(lobQuality)`). Which strand runs is decided elsewhere: `flow()` tries the active strands best-bound first, and the direct strand runs whenever the bridge ahead of it is dry, rejected or spent. The ...

- **fix(xrpl-ledger): F212 — a bridged crossing's round is bounded by the deferred line** (72af27a, xrpl-validator) The direct pool strand's `DirectStep` is bounded like every other: `PaymentSandbox::balanceHookIOU = min(live line, original - Σdebits)` (finding 197 for the exhausting book fills; finding 208 for the direct walk's pool turns). The bridged crossing's pool turn — the direct pool competing with the bridge each round — handed the pool the walk's own remainder instead, so the slice that exhausts the crossing's funds sold ...
- **fix(xrpl-ledger): F213 — a bridged crossing's remainder after a pool slice is sixteen digits** (a6bd0eb, xrpl-validator) A crossing's remaining in is an `STAmount`; rippled's `remainingIn` is an `IOUAmount` and the `DirectStep`'s bound a line balance, both sixteen digits (finding 118 keeps the book fills' remainder that way). The bridged crossing's pool turn kept the exact difference instead.
- **fix(xrpl-ledger): F214 — the bridged round's deferred bound is per iteration** (fd432ed, xrpl-validator) The deferred bound (finding 212) is per iteration: rippled's strand keeps its own `remainingIn` and the `DirectStep` re-reads the line and the deferred table each pass. Persisting the bound into the walk's remaining in left the chain a unit short of the live line from the first bounded round on, and at the exhausting round the chain undercut the deferred bound itself. The chain now continues from what the round ...
- **fix(xrpl-ledger): F215 — a bridged crossing's strands die for the rest of the crossing** (70d0675, xrpl-validator) rippled's `ActiveStrands` (`StrandFlow.h`) rebuilds the next iteration's strand list only from the strands that produced this iteration (`push`) and the untried remainder behind the winner (`pushRemainingCurToNext`); `activateNext` drops a pending strand whose upper bound misses `limitQuality` when more than one is pending, and never re-checks a lone one. A strand dropped or run dry is gone for the rest of the crossing, ...
- **fix(xrpl-ledger): F216 — a sell-mode offer crossing never re-anchors to its TakerPays** (bb922f1, xrpl-validator) Finding 210's forward re-anchor — deliver the reverse pass's cached out when the input that requires equals the input provided — has no cached out to re-anchor to in a `SELL` crossing. `CreateOffer::flowCross` replaces a sell's deliver amount with the largest representable one ("we are selling, so we will accept *more* than the offer specified", `OfferCreate.cpp`:416-441), so the reverse pass's `cache_ -> out` is ...
- **fix(xrpl-ledger): F217 — the payment pot counts down from SendMax; hop zero is bounded by the holding** (1e805e5, xrpl-validator) rippled's `remainingIn` counts down from `SendMax` — `sendMax - sum(savedIns)` (`StrandFlow.h`) — and the sender's holding enters only through the `DirectStep`'s per-iteration bound, `balanceHookIOU = min(live line, original - Σdebits)`. The payment engine clamped its pot to the holding up front, so the fold's remainder, not the line, sized the exhausting fill.
- **fix(xrpl-ledger): F218 — a spend that exactly meets the reverse pass runs no forward pass** (7163cdc, xrpl-validator) `flow()` (`StrandFlow.h`) walks a strand forward only from its limiting step. When the reverse pass finds nothing limiting — every step meets its request and the strand's in does not exceed `SendMax` — there is no forward pass at all: the reverse-sized amounts, already applied in the reverse pass's sandbox, are the strand's result. Our payment engine treated hop 0 spending its whole input as the mark of an input-limited ...
- **fix(xrpl-ledger): F219 — hop zero's gross cap is bounded by the holding** (65a988a, xrpl-validator) Finding 217 let the payment pot count down from `SendMax` and bounded hop zero by the sender's holding — but the holding is `GROSS` and the hop's budget is `NET` of the spend leg's transfer rate, so on a rated leg the bound never bit, and the verbatim gross cap the exhausting fill debits (finding 118) was `SendMax` itself.

- **fix(xrpl-ledger): F220 — an input-driven strand's last book hop consumes its carry** (3b2f993 , xrpl-validator) An input-limited strand's forward pass drives every book step by input, the last one included: fwdImp hands limitStepIn the whole carry, and the closing DirectStep clamps the delivery to the reverse want (setCacheLimiting , finding 170), the excess redeemed against the issuer. Finding 204 walked the middle hops that way but kept the last hop out-limited to its want, so a last CLOB fill stopped short of the ...
- **fix(xrpl-ledger): F222 — AccountDelete refuses NFT holders and issuers before the directory walk** (e1e82e7 , xrpl-validator) AccountDelete.cpp:256-303 decides four things before it reads the owner directory, and we modelled only the Sequence rule — after the walk: MintedNFTokens != BurnedNFTokens → techHAS_OBLIGATIONS any NFTTokenPage in [nftokenPageMin, nftokenPageMax] → techHAS_OBLIGATIONS Sequence + 255 > view.seq() → tecTOO_SOON FirstNFTTokenSequence + MintedNFTokens + 255 > seq → tecTOO_SOON
- **fix(xrpl-ledger): F221 — a deposit refuses tecFROZEN when the issuer froze the pool's own line** (50e32c7 , xrpl-validator) AMMDeposit.cpp:330-346 checkAmount , while fixCleanup3_3_0 is not live: after the strong requireAuth on each deposited amount, rippled runs checkFrozen(view, ammAccountID, asset) — the AMM account's own trust line frozen by the issuer (or a global freeze) refuses the deposit tecFROZEN even though the depositor's line is clean; withdrawals stay allowed. Our preclaim ran the depositor's requireAuth/checkFrozen for ...
- **fix(xrpl-ledger): F223 — a strand-trimmed ask keeps the limit-quality forgiveness in the pool judge** (3f67177 , xrpl-validator) StrandFlow.h:664-672 trims remainingOut to the quality function's solve and remembers that it did (adjustedRemOut); the post-check at :739-741 then forgives a fill up to 1e-7 over limitQuality when — and only when — the request was trimmed. Our payment driver does the trim (adjusted_ask) and its own strand judge honours it, but the AMM pool judge in consume recomputed the solve, found the incoming request ...
- **fix(xrpl-ledger): F224 — the bridged strand's limit solve carries the in-side transfer fee** (9771135 , xrpl-validator) BookStep::getQualityFunc folds the in-side transfer fee into the first leg's quality function: adjustQualityWithFees(kQOne, prevStepDir) with the taker's DirectStep redeeming makes trIn the gets issuer's TransferRate, and that quality is combined as a CLOB-like constant BEFORE the pool curve (a CLOB tip gets the fee composed into its quality — the same product). Our bridged crossing composed the two pool curves ...
- **fix(xrpl-ledger): F225 — AMMWithdraw refuses tecAMM_BALANCE by rippled's pool-balance guards** (d4965d4 , xrpl-validator) AMMWithdraw.cpp:568-593 withdraw() judges the ACTUAL amounts of every mode before anything moves and refuses tecAMM_BALANCE when one side of the pool would be drained but not the other, when every LP token is burned without draining both sides, or when more than the pool holds of either asset would leave. A one-asset withdrawal carries no second amount, and rippled's std::optional compares unequal to any balance, ...
- **fix(xrpl-ledger): F226 — the exhausting fill's remainder folds the saved inputs as a sorted multiset** (8324b31 , xrpl-validator) flow() recomputes remainingIn = sendMax - sum(savedIns) after every iteration (StrandFlow.h:745), and savedIns is a sorted multiset: the sixteen-digit fold runs in ASCENDING order — the same shape finding 202 gave savedOuts for remainingOut. Our crossing walks rebuilt the exhausting fill's remainder from in_gross_spent , an insertion-order Σ , and the two folds round apart once in a while.

- **fix(loop): runner diff mode rebuilds differential_probe unconditionally** (798c6c8 , xrpl-validator) cmd_diff built the native-leg binary only when it was absent, so loop/harness/target/debug/differential_probe sat at 2026-09-05 23:06 while the harness tree tracked main — the hourly scout judged every "diff" probe with a two-day-old engine. On 2026-09-07 that produced four phantom AMMWithdraw tecAMM_INVALID_TOKENS finds (#106816232 D45A17916FD1, #106821332 A56150E02425, #106822082 B857D3D4FF66, #106826178 ...
- **fix(loop): differential_probe hydrates the pool directory/lines and the taker's pays-side line** (248bb47 , xrpl-validator) Two hydration gaps in the scout's native leg, each a phantom TER refusal on state the live shadow and probe_bundle judge correctly: - AMMDeposit/AMMWithdraw: load_amm_prestate fetched only the pool's directory root and tail page and none of the other LPs' lines, so the engine's isOnlyLiquidityProvider walk (verify_and_adjust_lp_token_balance skips unhydrated entries) saw a sole LP, snapped the line against ...
- **fix(scripts): fetch_tx_bundle pulls the escrow, channel and check a refused tx names** (bd09003 , xrpl-validator) A tec EscrowFinish/EscrowCancel (or a refused PaymentChannel/Check claim) touches nothing but the fee, so the object it names never reaches the meta and the bundle lacked it; the probe then answered tecNO_TARGET for whatever mainnet actually decided. #106743443 44A81646DF43: an EscrowFinish two seconds before FinishAfter — mainnet tecNO_PERMISSION, ours tecNO_TARGET with the escrow unhydrated. keylet::escrow = ...

Tests and campaigns

- **test(xrpl-node): close the F201 vector test — the F201/F202 merge resolution dropped its brace** (0c4d063 , xrpl-validator)
- **test(xrpl-node): fetch_tx_bundle rebuilds later-touched targets from the tx's own meta** (bafa4fd , xrpl-validator)
- **test(xrpl-node): fetch_tx_bundle rebuilds from FinalFields alone** (9795101 , xrpl-validator)

Docs and operations

- **chore(xrpl-ledger): drop the dead initializer clippy flagged on pool_bq_blocked** (26c3b63 , xrpl-validator)
- **chore(xrpl-ledger): markdown bullets in the F225 guard doc (clippy doc_lazy_continuation)** (5d875fe , xrpl-validator)

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_3_0	2026-08-28 11:15 UTC	2026-09-11 11:15 UTC

Amendments enabled on mainnet: 93.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Scan-Key Stealth Payment Destinations** (privacy) Add an optional sfStealthScanKey (33-byte compressed pubkey) to AccountRoot, settable via a new AccountSet flag asfStealthAddress, and add a Payment f
- **Concentrated AMM Ranges with Fee Tiers** (amm-defi) Add a new owner-reserved ledger object ltAMM_POSITION (keylet::ammPosition(ammID, owner, seq)) holding sfTickLower, sfTickUpper (int32 price ticks on
- **Header-Proof Cross-Chain Message Bridge** (interop) Extend the Bridge object with an ltXCHAIN_HEADER_STORE tracking sfLastVerifiedLedgerHash/Seq and sfCounterpartyUNLHash, populated by a new XChainHeade

Appendix — commits

Time	Repo	Commit	Subject
00:30	xrpl-validator	c1b1e51	fix(xrpl-ledger): F198 — the direct walk's tail pool turn iterates like rippled's flow()
01:16	xrpl-validator	79395eb	fix(xrpl-ledger): F199 — TrustSet refuses tfSetNoRipple on a negative balance
01:57	xrpl-validator	a8439a9	fix(xrpl-ledger): F200 — an in-limited pool slice delivers no more than the remaining want
03:49	xrpl-validator	7c49aa8	fix(xrpl-ledger): F201 — consecutive self-offers are swept in the pass that is still walking
04:34	xrpl-validator	5182de9	fix(xrpl-ledger): F202 — the bridged walk's remaining want is the entry want less the ascending fold of its saved outs
04:43	xrpl-validator	0c4d063	test(xrpl-node): close the F201 vector test — the F201/F202 merge resolution dropped its brace
04:43	xrpl-validator	7aaf827	fix(xrpl-ledger): F206 — a pool slice's quality is getRate-rounded, as rippled's Quality{amounts}
04:43	xrpl-validator	2b27ffd	fix(xrpl-ledger): F205 — a bridge admitted by its multipath bound runs only if its single-path bound clears the limit too
04:43	xrpl-validator	888ecec	fix(xrpl-ledger): F204 — an input-limited payment strand drives its middle hops by the carry
04:43	xrpl-validator	955de48	fix(xrpl-ledger): F203 — a payment strand's quality function takes a pool only where its fee-inclusive spot beats the tip
08:48	xrpl-validator	bdc467e	fix(xrpl-ledger): F208 — a crossing's pool turn is bounded by the taker's live line
09:14	xrpl-validator	f0642ec	fix(xrpl-ledger): F209 — an IOU difference at Number's cusp rounds the coarse mantissa
09:16	xrpl-validator	a2b29db	fix(xrpl-ledger): F210 — a forward pool hop re-anchors to the reverse pass's cached out
09:56	xrpl-validator	b5d35c7	fix(xrpl-ledger): F211 — a dry rival strand hands the round to the direct pool
10:05	xrpl-validator	72af27a	fix(xrpl-ledger): F212 — a bridged crossing's round is bounded by the deferred line
10:08	xrpl-validator	a6bd0eb	fix(xrpl-ledger): F213 — a bridged crossing's remainder after a pool slice is sixteen digits

Time	Repo	Commit	Subject
10:17	xrpl-validator	fd432ed	fix(xrpl-ledger): F214 — the bridged round's deferred bound is per iteration
10:31	xrpl-validator	70d0675	fix(xrpl-ledger): F215 — a bridged crossing's strands die for the rest of the crossing
10:49	xrpl-validator	bb922f1	fix(xrpl-ledger): F216 — a sell-mode offer crossing never re-anchors to its TakerPays
10:56	xrpl-validator	1e805e5	fix(xrpl-ledger): F217 — the payment pot counts down from SendMax; hop zero is bounded by the holding
11:08	xrpl-validator	7163cdc	fix(xrpl-ledger): F218 — a spend that exactly meets the reverse pass runs no forward pass
11:14	xrpl-validator	bafa4fd	test(xrpl-node): fetch_tx_bundle rebuilds later-touched targets from the tx's own meta
12:38	xrpl-validator	9795101	test(xrpl-node): fetch_tx_bundle rebuilds from FinalFields alone
12:41	xrpl-validator	65a988a	fix(xrpl-ledger): F219 — hop zero's gross cap is bounded by the holding
12:44	xrpl-validator	3b2f993	fix(xrpl-ledger): F220 — an input-driven strand's last book hop consumes its carry
12:47	xrpl-validator	26c3b63	chore(xrpl-ledger): drop the dead initializer clippy flagged on pool_bq_blocked
14:04	xrpl-validator	798c6c8	fix(loop): runner diff mode rebuilds differential_probe unconditionally
14:16	xrpl-validator	248bb47	fix(loop): differential_probe hydrates the pool directory/lines and the taker's pays-side line
14:30	xrpl-validator	bd09003	fix(scripts): fetch_tx_bundle pulls the escrow, channel and check a refused tx names
14:30	xrpl-validator	e1e82e7	fix(xrpl-ledger): F222 — AccountDelete refuses NFT holders and issuers before the directory walk
14:30	xrpl-validator	50e32c7	fix(xrpl-ledger): F221 — a deposit refuses tecFROZEN when the issuer froze the pool's own line
18:16	xrpl-validator	3f67177	fix(xrpl-ledger): F223 — a strand-trimmed ask keeps the limit-quality forgiveness in the pool judge
18:37	xrpl-validator	9771135	fix(xrpl-ledger): F224 — the bridged strand's limit solve carries the in-side transfer fee
19:56	xrpl-validator	d4965d4	fix(xrpl-ledger): F225 — AMMWithdraw refuses tecAMM_BALANCE by rippled's pool-balance guards
20:07	xrpl-validator	5d875fe	chore(xrpl-ledger): markdown bullets in the F225 guard doc (clippy doc_lazy_continuation)
22:47	xrpl-validator	8324b31	fix(xrpl-ledger): F226 — the exhausting fill's remainder folds the saved inputs as a sorted multiset